

CHƯƠNG 3. PHÂN LOẠI

Trong Chương 1, tôi đã đề cập rằng các tác vụ học có giám sát phổ biến nhất là hồi quy (dự đoán giá trị) và phân loại (dự đoán lớp). Trong Chương 2, chúng ta đã khám phá một tác vụ hồi quy, dự đoán giá trị nhà ở, sử dụng các thuật toán khác nhau như hồi quy tuyến tính, cây quyết định và rừng ngẫu nhiên (sẽ được giải thích chi tiết hơn trong các chương sau). Bây giờ chúng ta sẽ chuyển sự chú ý sang các hệ thống phân loại.

3.1 MNIST

Trong chương này, chúng ta sẽ sử dụng bộ dữ liệu MNIST, đây là một tập hợp 70.000 hình ảnh nhỏ của các chữ số viết tay bởi học sinh trung học và nhân viên Cục Thống kê Hoa Kỳ. Mỗi hình ảnh được gắn nhãn với chữ số mà nó đại diện. Tập hợp này đã được nghiên cứu nhiều đến nỗi nó thường được gọi là “hello world” của học máy: bất cứ khi nào mọi người đưa ra một thuật toán phân loại mới, họ đều tò mò muốn xem nó sẽ hoạt động như thế nào trên MNIST, và bất cứ ai học học máy đều xử lý bộ dữ liệu này sớm hay muộn.

Scikit-Learn cung cấp nhiều hàm trợ giúp để tải xuống các bộ dữ liệu phổ biến. MNIST là một trong số đó. Đoạn mã sau lấy bộ dữ liệu MNIST từ OpenML.org:

```
from sklearn.datasets import fetch_openml

mnist = fetch_openml('mnist_784', as_frame=False)
```

Gói `sklearn.datasets` chủ yếu chứa ba loại hàm: các hàm `fetch_*` như `fetch_openml()` để tải xuống các bộ dữ liệu thực tế, các hàm `load_*` để tải các bộ dữ liệu đồ chơi nhỏ được đóng gói cùng với Scikit-Learn (vì vậy chúng không cần được tải xuống qua internet), và các hàm `make_*` để tạo các bộ dữ liệu giả, hữu ích cho các kiểm thử. Các bộ dữ liệu được tạo thường được trả về dưới dạng một bộ (X, y) chứa dữ liệu đầu vào và các mục tiêu, cả hai đều dưới dạng mảng NumPy. Các bộ dữ liệu khác được trả về dưới dạng đối tượng `sklearn.utils.Bunch`, đây là các từ điển mà các mục nhập của chúng cũng có thể được truy cập dưới dạng thuộc tính. Chúng thường chứa các mục nhập sau:

- “DESCR”: Mô tả của bộ dữ liệu
- “data”: Dữ liệu đầu vào, thường là mảng NumPy 2D
- “target”: Các nhãn, thường là mảng NumPy 1D

Hàm `fetch_openml()` hơi bất thường vì theo mặc định, nó trả về đầu vào dưới dạng Pandas DataFrame và nhãn dưới dạng Pandas Series (trừ khi bộ dữ liệu thừa thớt). Nhưng bộ dữ liệu MNIST chứa hình ảnh, và DataFrame không lý tưởng cho điều đó, vì vậy tốt hơn là đặt `as_frame=False` để lấy dữ liệu dưới dạng mảng NumPy. Hãy xem các mảng này:

```
>>> X, y = mnist.data, mnist.target
>>> X
array([[0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.]])
```

```

...,
[0., 0., 0., ..., 0., 0., 0.],
[0., 0., 0., ..., 0., 0., 0.],
[0., 0., 0., ..., 0., 0., 0.]])
>>> X.shape
(70000, 784)
>>> y
array(['5', '0', '4', ..., '4', '5', '6'], dtype=object)
>>> y.shape
(70000,)

```

Có 70.000 hình ảnh, và mỗi hình ảnh có 784 đặc trưng. Điều này là do mỗi hình ảnh là 28×28 pixel, và mỗi đặc trưng chỉ đơn giản đại diện cho cường độ của một pixel, từ 0 (trắng) đến 255 (đen). Hãy xem một chữ số từ bộ dữ liệu (Hình 3-1). Tất cả những gì chúng ta cần làm là lấy vector đặc trưng của một trường hợp, định hình lại nó thành một mảng 28×28 và hiển thị nó bằng hàm `imshow()` của Matplotlib. Chúng ta sử dụng `cmap="binary"` để có bản đồ màu xám trong đó 0 là trắng và 255 là đen:

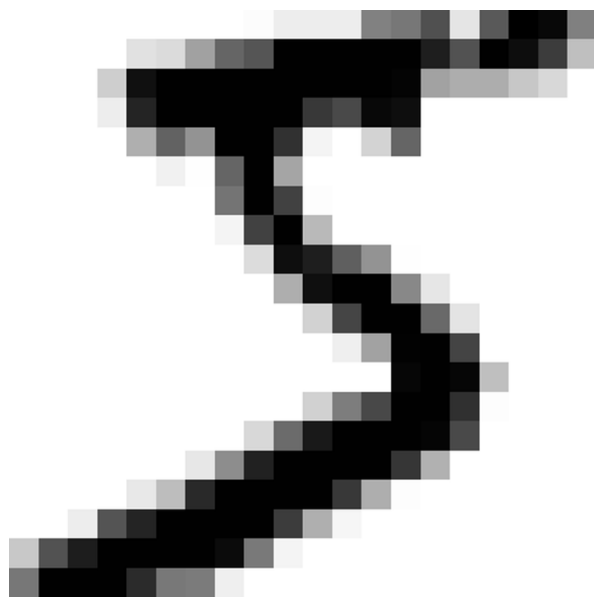
```

import matplotlib.pyplot as plt

def plot_digit(image_data):
    image = image_data.reshape(28, 28)
    plt.imshow(image, cmap="binary")
    plt.axis("off")

some_digit = X[0]
plot_digit(some_digit)
plt.show()

```



Hình 3-1. Ví dụ về hình ảnh MNIST

Điều này trông giống số 5, và đúng là nhãn cũng cho chúng ta biết điều đó:

```
>>> y[0]
'5'
```

Để bạn cảm nhận được sự phức tạp của tác vụ phân loại, Hình 3-2 cho thấy một vài hình ảnh khác từ bộ dữ liệu MNIST.

Nhưng chờ đã! Bạn nên luôn tạo một tập kiểm thử và đặt nó sang một bên trước khi kiểm tra dữ liệu kỹ lưỡng. Bộ dữ liệu MNIST được trả về bởi `fetch_openml()` thực tế đã được chia thành một tập huấn luyện (60.000 hình ảnh đầu tiên) và một tập kiểm thử (10.000 hình ảnh cuối cùng):

```
X_train, X_test, y_train, y_test = X[:60000], X[60000:], y[:60000], y[60000:]
```

Tập huấn luyện đã được xáo trộn sẵn cho chúng ta, điều này tốt vì nó đảm bảo rằng tất cả các fold kiểm định chéo sẽ tương tự nhau (chúng ta không muốn một fold bị thiếu một số chữ số). Hơn nữa, một số thuật toán học nhạy cảm với thứ tự của các trường hợp huấn luyện, và chúng hoạt động kém nếu chúng nhận được nhiều trường hợp tương tự liên tiếp. Việc xáo trộn bộ dữ liệu đảm bảo rằng điều này sẽ không xảy ra.



Hình 3-2. Các chữ số từ bộ dữ liệu MNIST

3.2 Huấn luyện bộ phân loại nhị phân

Bây giờ, hãy đơn giản hóa vấn đề và chỉ cố gắng xác định một chữ số — ví dụ, số 5. “Bộ phát hiện 5” này sẽ là một ví dụ về bộ phân loại nhị phân, có khả năng phân biệt giữa chỉ hai lớp, 5 và không-5. Đầu tiên, chúng ta sẽ tạo các vector mục tiêu cho tác vụ phân loại này:

```
y_train_5 = (y_train == '5') # True for all 5s, False for all other digits
y_test_5 = (y_test == '5')
```

Bây giờ hãy chọn một bộ phân loại và huấn luyện nó. Một nơi tốt để bắt đầu là với bộ phân loại xuống dốc ngẫu nhiên (SGD, hoặc SGD ngẫu nhiên), sử dụng lớp

SGDClassifier của Scikit-Learn.

Bộ phân loại này có khả năng xử lý các tập dữ liệu rất lớn một cách hiệu quả. Một phần là do SGD xử lý các trường hợp huấn luyện độc lập, từng trường hợp một, điều này cũng làm cho SGD rất phù hợp cho học trực tuyến, như bạn sẽ thấy sau này. Hãy tạo một SGDClassifier và huấn luyện nó trên toàn bộ tập huấn luyện:

```
from sklearn.linear_model import SGDClassifier

sgd_clf = SGDClassifier(random_state=42)
sgd_clf.fit(X_train, y_train_5)
```

Bây giờ chúng ta có thể sử dụng nó để phát hiện hình ảnh của số 5:

```
>>> sgd_clf.predict([some_digit])
array([ True])
```

Bộ phân loại đoán rằng hình ảnh này đại diện cho một số 5 (True). Có vẻ như nó đã đoán đúng trong trường hợp cụ thể này! Bây giờ, hãy đánh giá hiệu suất của mô hình này.

3.3 Các thước đo hiệu suất

Đánh giá một bộ phân loại thường phức tạp hơn đáng kể so với đánh giá một bộ hồi quy, vì vậy chúng ta sẽ dành một phần lớn chương này cho chủ đề này. Có nhiều thước đo hiệu suất có sẵn, vì vậy hãy pha một ly cà phê nữa và sẵn sàng tìm hiểu một loạt các khái niệm và từ viết tắt mới!

3.3.1 Đo độ chính xác bằng kiểm định chéo

Một cách hay để đánh giá một mô hình là sử dụng kiểm định chéo, giống như bạn đã làm trong Chương 2. Hãy sử dụng hàm `cross_val_score()` để đánh giá mô hình SGDClassifier của chúng ta, sử dụng kiểm định chéo k-fold với ba fold. Nhớ rằng kiểm định chéo k-fold có nghĩa là chia tập huấn luyện thành k fold (trong trường hợp này là ba), sau đó huấn luyện mô hình k lần, mỗi lần giữ lại một fold khác để đánh giá (xem Chương 2):

```
>>> from sklearn.model_selection import cross_val_score
```

```
>>> cross_val_score(sgd_clf, X_train, y_train_5, cv=3, scoring="accuracy")
array([0.95035, 0.96035, 0.9604 ])
```

Wow! Độ chính xác (tỷ lệ dự đoán đúng) trên 95% trên tất cả các fold kiểm định chéo? Điều này có vẻ tuyệt vời, phải không? Chà, trước khi bạn quá phấn khích, hãy xem một bộ phân loại giả (dummy classifier) mà chỉ phân loại mọi hình ảnh duy nhất vào lớp thường xuyên nhất, trong trường hợp này là lớp phủ định (tức là không phải 5):

```
from sklearn.dummy import DummyClassifier

dummy_clf = DummyClassifier()
dummy_clf.fit(X_train, y_train_5)
print(any(dummy_clf.predict(X_train))) # prints False: no 5s detected
```

Bạn có thể đoán độ chính xác của mô hình này không? Hãy cùng tìm hiểu:

```
>>> cross_val_score(dummy_clf, X_train, y_train_5, cv=3, scoring="accuracy")
array([0.90965, 0.90965, 0.90965])
```

Đúng vậy, nó có độ chính xác trên 90%! Điều này đơn giản là vì chỉ khoảng 10% hình ảnh là số 5, vì vậy nếu bạn luôn đoán rằng một hình ảnh không phải là số 5, bạn sẽ đúng khoảng 90% thời gian. Điều này vượt qua Nostradamus.

Điều này chứng tỏ tại sao độ chính xác nói chung không phải là thước đo hiệu suất được ưu tiên cho các bộ phân loại, đặc biệt khi bạn đang xử lý các tập dữ liệu bị lệch (tức là khi một số lớp thường xuyên hơn nhiều so với các lớp khác). Một cách tốt hơn nhiều để đánh giá hiệu suất của bộ phân loại là xem xét ma trận nhầm lẫn (CM).

THỰC HIỆN KIỂM ĐỊNH CHÉO Đôi khi bạn sẽ cần kiểm soát nhiều hơn đối với quy trình kiểm định chéo so với những gì Scikit-Learn cung cấp sẵn. Trong những trường hợp này, bạn có thể tự mình thực hiện kiểm định chéo. Đoạn mã sau đây thực hiện gần giống với hàm `cross_val_score()` của Scikit-Learn, và nó in ra cùng kết quả:

```
from sklearn.model_selection import StratifiedKFold
from sklearn.base import clone

skfolds = StratifiedKFold(n_splits=3) # add shuffle=True if the dataset is
                                     # not already shuffled
for train_index, test_index in skfolds.split(X_train, y_train_5):
    clone_clf = clone(sgd_clf)
    X_train_folds = X_train[train_index]
    y_train_folds = y_train_5[train_index]
    X_test_fold = X_train[test_index]
    y_test_fold = y_train_5[test_index]

    clone_clf.fit(X_train_folds, y_train_folds)
    y_pred = clone_clf.predict(X_test_fold)
    n_correct = sum(y_pred == y_test_fold)
    print(n_correct / len(y_pred)) # prints 0.95035, 0.96035, and 0.9604
```

Lớp StratifiedKFold thực hiện lấy mẫu phân tầng (như đã giải thích trong Chương 2) để tạo ra các fold chứa tỷ lệ đại diện của mỗi lớp. Ở mỗi lần lặp, mã tạo ra một bản sao của bộ phân loại, huấn luyện bản sao đó trên các fold huấn luyện và đưa ra dự đoán trên fold kiểm thử. Sau đó, nó đếm số lượng dự đoán đúng và xuất ra tỷ lệ dự đoán đúng.

3.3.2 Ma trận nhầm lẫn

Ý tưởng chung của ma trận nhầm lẫn là đếm số lần các trường hợp của lớp A được phân loại là lớp B, cho tất cả các cặp A/B. Ví dụ, để biết số lần bộ phân loại nhầm lẫn hình ảnh số 8 với số 0, bạn sẽ xem hàng #8, cột #0 của ma trận nhầm lẫn.

Để tính toán ma trận nhầm lẫn, trước tiên bạn cần có một tập hợp các dự đoán để có thể so sánh chúng với các mục tiêu thực tế. Bạn có thể đưa ra dự đoán trên tập kiểm thử, nhưng tốt nhất là giữ nguyên tập đó tạm thời (hãy nhớ rằng bạn chỉ muốn sử dụng tập kiểm thử vào cuối dự án của mình, một khi bạn đã có một bộ phân loại mà bạn sẵn sàng triển khai). Thay vào đó, bạn có thể sử dụng hàm `cross_val_predict()`:

```
from sklearn.model_selection import cross_val_predict
y_train_pred = cross_val_predict(sgd_clf, X_train, y_train_5, cv=3)
```

Giống như hàm `cross_val_score()`, `cross_val_predict()` thực hiện kiểm định chéo k-fold, nhưng thay vì trả về các điểm đánh giá, nó trả về các dự đoán được đưa ra trên mỗi fold kiểm thử. Điều này có nghĩa là bạn nhận được một dự đoán sạch cho mỗi trường hợp trong tập huấn luyện (bởi “sạch” tôi muốn nói “ngoài mẫu”: mô hình đưa ra dự đoán trên dữ liệu mà nó chưa từng thấy trong quá trình huấn luyện).

Bây giờ bạn đã sẵn sàng lấy ma trận nhầm lẫn bằng cách sử dụng hàm `confusion_matrix()`. Chỉ cần chuyển cho nó các lớp mục tiêu (`y_train_5`) và các lớp được dự đoán (`y_train_pred`):

```
>>> from sklearn.metrics import confusion_matrix
>>> cm = confusion_matrix(y_train_5, y_train_pred)
>>> cm
array([[53892,   687],
       [ 1891,  3530]])
```

Mỗi hàng trong ma trận nhầm lẫn biểu thị một lớp thực tế, trong khi mỗi cột biểu thị một lớp được dự đoán. Hàng đầu tiên của ma trận này xem xét các hình ảnh không phải là 5 (lớp phủ định): 53.892 trong số đó được phân loại đúng là không phải 5 (chúng được gọi là true negatives), trong khi 687 còn lại bị phân loại sai là 5 (false positives, còn gọi là lỗi loại I). Hàng thứ hai xem xét các hình ảnh số 5 (lớp tích cực): 1.891 bị phân loại sai là không phải 5 (false negatives, còn gọi là lỗi loại II), trong khi 3.530 còn lại được phân loại đúng là số 5 (true positives). Một bộ phân loại hoàn hảo sẽ chỉ có true positives và true negatives, vì vậy ma trận nhầm lẫn của nó sẽ chỉ có các giá trị khác 0 trên đường chéo chính của nó (từ trên cùng bên trái xuống dưới cùng bên phải):

```
>>> y_train_perfect_predictions = y_train_5 # pretend we reached perfection
>>> confusion_matrix(y_train_5, y_train_perfect_predictions)
```

```
array([[54579, 0],
       [0, 5421]])
```

Ma trận nhầm lẫn cung cấp cho bạn rất nhiều thông tin, nhưng đôi khi bạn có thể muốn một chỉ số ngắn gọn hơn. Một chỉ số thú vị đáng xem xét là độ chính xác của các dự đoán tích cực; đây được gọi là độ chính xác (precision) của bộ phân loại (Phương trình 3-1).

Phương trình 3-1. Độ chính xác

$$precision = \frac{TP}{TP + FP}$$

TP là số lượng true positives, và FP là số lượng false positives.

Một cách đơn giản để có độ chính xác hoàn hảo là tạo một bộ phân loại luôn đưa ra dự đoán tiêu cực, ngoại trừ một dự đoán tích cực duy nhất trên trường hợp mà nó tự tin nhất. Nếu dự đoán này là đúng, thì bộ phân loại có độ chính xác 100% (precision = 1/1 = 100%). Rõ ràng, một bộ phân loại như vậy sẽ không hữu ích lắm, vì nó sẽ bỏ qua tất cả ngoại trừ một trường hợp tích cực. Vì vậy, độ chính xác thường được sử dụng cùng với một chỉ số khác có tên là độ nhạy (recall), còn được gọi là độ nhạy (sensitivity) hoặc tỷ lệ dương tính đúng (TPR): đây là tỷ lệ các trường hợp tích cực được bộ phân loại phát hiện đúng (Phương trình 3-2).

Phương trình 3-2. Độ nhạy

$$recall = \frac{TP}{TP + FN}$$

FN tất nhiên là số lượng false negatives.

Nếu bạn bối rối về ma trận nhầm lẫn, Hình 3-3 có thể giúp ích.

		Predicted		
		Negative	Positive	
Actual	Negative	8 3 9	6	Precision (e.g., 3 out of 4)
	Positive	5 5	5 5 5	
		FN	TP	Recall (e.g., 3 out of 5)

Hình 3-3. Ma trận nhầm lẫn minh họa hiển thị các ví dụ về true negatives (trên cùng bên trái), false positives (trên cùng bên phải), false negatives (dưới cùng bên trái) và true positives (dưới cùng bên phải).

3.3.3 Độ chính xác và độ nhạy

Scikit-Learn cung cấp một số hàm để tính toán các chỉ số phân loại, bao gồm độ chính xác và độ nhạy:

```
>>> from sklearn.metrics import precision_score, recall_score
>>> precision_score(y_train_5, y_train_pred) # == 3530 / (687 + 3530)
0.8370879772350012
>>> recall_score(y_train_5, y_train_pred) # == 3530 / (1891 + 3530)
0.6511713705958311
```

Bây giờ bộ phát hiện số 5 của chúng ta không còn sáng bóng như khi chúng ta nhìn vào độ chính xác của nó nữa. Khi nó cho rằng một hình ảnh đại diện cho số 5, nó chỉ đúng 83.7% thời gian. Hơn nữa, nó chỉ phát hiện được 65.1% số 5.

Thường thì thuận tiện để kết hợp độ chính xác và độ nhạy vào một chỉ số duy nhất được gọi là điểm F1, đặc biệt khi bạn cần một chỉ số duy nhất để so sánh hai bộ phân loại. Điểm F1 là trung bình điều hòa của độ chính xác và độ nhạy (Phương trình 3-3). Trong khi trung bình thông thường coi tất cả các giá trị như nhau, trung bình điều hòa lại ưu tiên các giá trị thấp hơn nhiều. Kết quả là, bộ phân loại sẽ chỉ đạt được điểm F1 cao nếu cả độ nhạy và độ chính xác đều cao.

Phương trình 3-3. Điểm F1

$$F_1 = \frac{2}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}} = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} = \frac{TP}{TP + \frac{FN+FP}{2}}$$

Để tính điểm F1, chỉ cần gọi hàm `f1_score()`:

```
>>> from sklearn.metrics import f1_score
>>> f1_score(y_train_5, y_train_pred)
0.7325171197343846
```

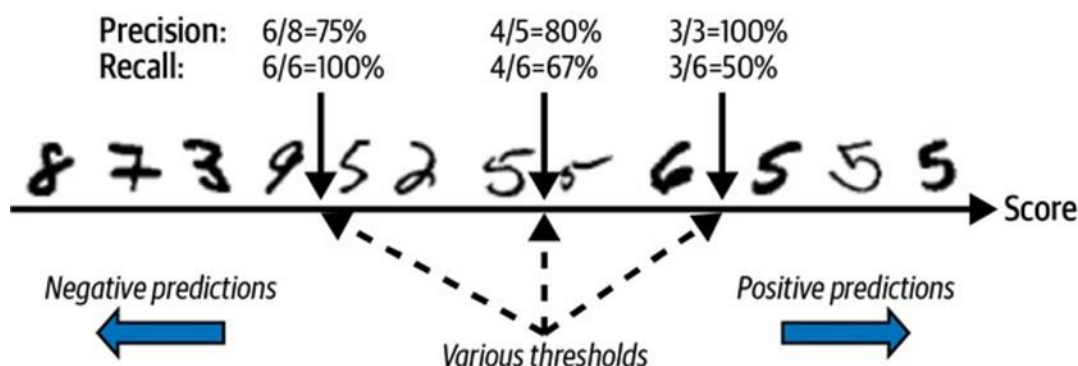
Điểm F1 ưu tiên các bộ phân loại có độ chính xác và độ nhạy tương tự nhau. Điều này không phải lúc nào cũng là điều bạn muốn: trong một số ngữ cảnh, bạn chủ yếu quan tâm đến độ chính xác, và trong các ngữ cảnh khác, bạn thực sự quan tâm đến độ nhạy. Ví dụ, nếu bạn huấn luyện một bộ phân loại để phát hiện các video an toàn cho trẻ em, bạn có thể sẽ thích một bộ phân loại từ chối nhiều video tốt (độ nhạy thấp) nhưng chỉ giữ lại những video an toàn (độ chính xác cao), hơn là một bộ phân loại có độ nhạy cao hơn nhiều nhưng lại để một vài video thực sự tệ xuất hiện trong sản phẩm của bạn (trong những trường hợp như vậy, bạn thậm chí có thể muốn thêm một pipeline thủ công để kiểm tra việc lựa chọn video của bộ phân loại). Mặt khác, giả sử bạn huấn luyện một bộ phân loại để phát hiện những kẻ trộm trong hình ảnh giám sát: có lẽ ổn nếu bộ phân loại của bạn chỉ có độ chính xác 30% miễn là

nó có độ nhạy 99% (chắc chắn, nhân viên bảo vệ sẽ nhận được một vài cảnh báo sai, nhưng gần như tất cả những kẻ trộm sẽ bị bắt).

Thật không may, bạn không thể có cả hai: tăng độ chính xác sẽ làm giảm độ nhạy, và ngược lại. Điều này được gọi là sự đánh đổi độ chính xác/độ nhạy (precision/recall trade-off).

3.3.4 Sự đánh đổi Độ chính xác/Độ nhạy

Để hiểu sự đánh đổi này, hãy xem cách SGDClassifier đưa ra các quyết định phân loại của nó. Đối với mỗi trường hợp, nó tính toán một điểm số dựa trên một hàm quyết định. Nếu điểm số đó lớn hơn một ngưỡng, nó gán trường hợp đó vào lớp dương; nếu không, nó gán vào lớp âm. Hình 3-4 cho thấy một vài chữ số được đặt từ điểm thấp nhất bên trái đến điểm cao nhất bên phải. Giả sử ngưỡng quyết định được đặt ở mũi tên trung tâm (giữa hai số 5): bạn sẽ tìm thấy 4 dự đoán đúng (số 5 thực tế) ở bên phải ngưỡng đó, và 1 dự đoán sai (thực tế là số 6). Do đó, với ngưỡng đó, độ chính xác là 80% (4 trên 5). Nhưng trong số 6 số 5 thực tế, bộ phân loại chỉ phát hiện 4, vì vậy độ nhạy là 67% (4 trên 6). Nếu bạn tăng ngưỡng (di chuyển nó đến mũi tên bên phải), dự đoán sai (số 6) trở thành một dự đoán đúng tiêu cực, do đó tăng độ chính xác (lên đến 100% trong trường hợp này), nhưng một dự đoán đúng tích cực trở thành một dự đoán sai tiêu cực, làm giảm độ nhạy xuống 50%. Ngược lại, việc giảm ngưỡng sẽ làm tăng độ nhạy và giảm độ chính xác.



Hình 3-4. Sự đánh đổi độ chính xác/độ nhạy: hình ảnh được xếp hạng theo điểm số bộ phân loại của chúng, và những hình ảnh trên ngưỡng quyết định được chọn được coi là tích cực; ngưỡng càng cao thì độ nhạy càng thấp, nhưng (nói chung) độ chính xác càng cao.

Scikit-Learn không cho phép bạn đặt trực tiếp ngưỡng, nhưng nó cung cấp cho bạn quyền truy cập vào các điểm quyết định mà nó sử dụng để đưa ra dự đoán. Thay vì gọi phương thức `predict()` của bộ phân loại, bạn có thể gọi phương thức `decision_function()` của nó, phương thức này trả về một điểm số cho mỗi trường hợp, và sau đó sử dụng bất kỳ ngưỡng nào bạn muốn để đưa ra dự đoán dựa trên các điểm số đó.

```
>>> y_scores = sgd_clf.decision_function([some_digit])
>>> y_scores
array([2164.22030239])
>>> threshold = 0
>>> y_some_digit_pred = (y_scores > threshold)
array([ True])
```

SGDClassifier sử dụng ngưỡng bằng 0, vì vậy đoạn mã trên trả về kết quả tương tự như phương thức `predict()` (tức là True). Hãy tăng ngưỡng:

```
>>> threshold = 3000
>>> y_some_digit_pred = (y_scores > threshold)
>>> y_some_digit_pred
array([False])
```

Điều này xác nhận rằng việc tăng ngưỡng làm giảm độ nhạy. Hình ảnh thực tế đại diện cho số 5, và bộ phân loại phát hiện nó khi ngưỡng là 0, nhưng nó bỏ sót khi ngưỡng tăng lên 3.000.

Làm thế nào để bạn quyết định sử dụng ngưỡng nào? Đầu tiên, sử dụng hàm `cross_val_predict()` để lấy điểm của tất cả các trường hợp trong tập huấn luyện, nhưng lần này chỉ định rằng bạn muốn trả về điểm quyết định thay vì dự đoán:

```
y_scores = cross_val_predict(sgd_clf, X_train, y_train_5, cv=3,
                             method="decision_function")
```

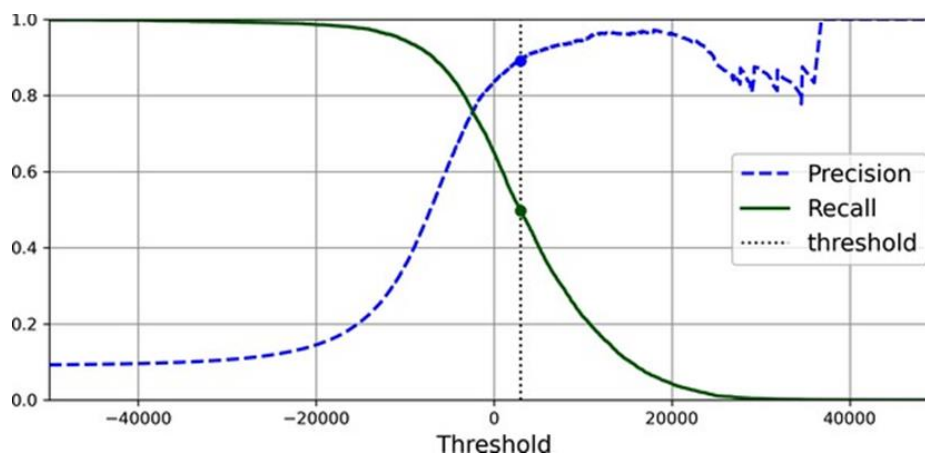
Với các điểm số này, sử dụng hàm `precision_recall_curve()` để tính toán độ chính xác và độ nhạy cho tất cả các ngưỡng có thể (hàm này thêm một độ chính xác cuối cùng là 0 và một độ nhạy cuối cùng là 1, tương ứng với một ngưỡng vô hạn):

```
from sklearn.metrics import precision_recall_curve

precisions, recalls, thresholds = precision_recall_curve(y_train_5, y_scores)
```

Cuối cùng, sử dụng Matplotlib để vẽ biểu đồ độ chính xác và độ nhạy dưới dạng hàm của giá trị ngưỡng (Hình 3-5). Hãy hiển thị ngưỡng 3.000 mà chúng ta đã chọn:

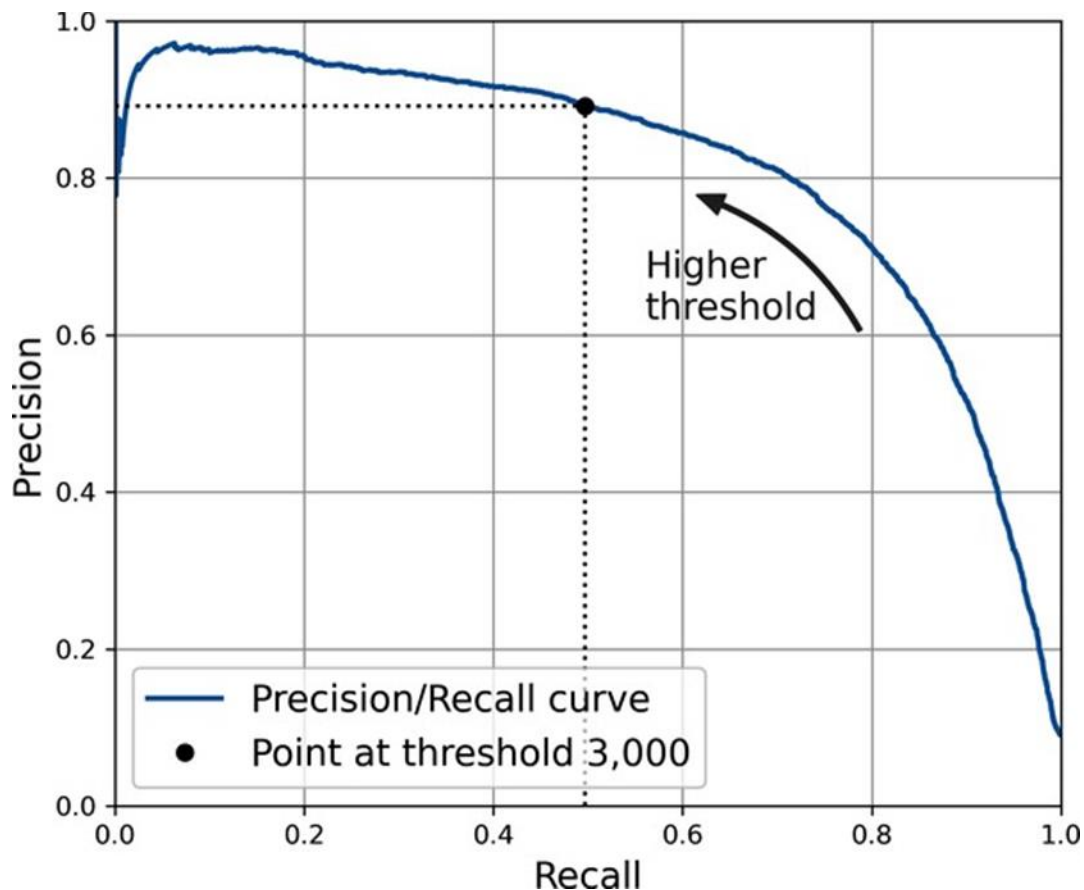
```
plt.plot(thresholds, precisions[:-1], "b--", label="Precision", linewidth=2)
plt.plot(thresholds, recalls[:-1], "g-", label="Recall", linewidth=2)
plt.vlines(threshold, 0, 1.0, "k", "dotted", label="threshold")
[...] # beautify the figure: add grid, legend, axis, labels, and circles
plt.show()
```



Hình 3-5. Độ chính xác và độ nhạy so với ngưỡng quyết định.

Tại giá trị ngưỡng này, độ chính xác gần 90% và độ nhạy khoảng 50%. Một cách khác để chọn một sự đánh đổi độ chính xác/độ nhạy tốt là vẽ trực tiếp độ chính xác so với độ nhạy, như được hiển thị trong Hình 3-6 (cùng ngưỡng được hiển thị).

```
plt.plot(recalls, precisions, linewidth=2, label="Precision/Recall curve")
[...] # beautify the figure: add labels, grid, legend, arrow, and text
plt.show()
```



Hình 3-6. Độ chính xác so với độ nhạy.

Bạn có thể thấy rằng độ chính xác thực sự bắt đầu giảm mạnh ở khoảng 80% độ nhạy. Bạn có lẽ sẽ muốn chọn một sự đánh đổi độ chính xác/độ nhạy ngay trước khi xảy ra sự giảm đó — ví dụ, ở khoảng 60% độ nhạy. Nhưng tất nhiên, lựa chọn phụ thuộc vào dự án của bạn.

Giả sử bạn quyết định đạt được độ chính xác 90%. Bạn có thể sử dụng biểu đồ đầu tiên để tìm ngưỡng bạn cần sử dụng, nhưng điều đó không chính xác lắm. Thay vào đó, bạn có thể tìm kiếm ngưỡng thấp nhất mang lại cho bạn ít nhất 90% độ chính xác. Đối với điều này, bạn có thể sử dụng phương thức `argmax()` của mảng NumPy. Phương thức này trả về chỉ mục đầu tiên của giá trị lớn nhất, trong trường hợp này có nghĩa là giá trị True đầu tiên.

```
>>> idx_for_90_precision = (precisions >= 0.90).argmax()
>>> threshold_for_90_precision = thresholds[idx_for_90_precision]
```

```
>>> threshold_for_90_precision
3370.0194991439557
```

Để đưa ra dự đoán (trên tập huấn luyện hiện tại), thay vì gọi phương thức `predict()` của bộ phân loại, bạn có thể chạy đoạn mã này:

```
y_train_pred_90 = (y_scores >= threshold_for_90_precision)
```

Hãy kiểm tra độ chính xác và độ nhạy của các dự đoán này:

```
>>> precision_score(y_train_5, y_train_pred_90)
0.9000345901072293
>>> recall_at_90_precision = recall_score(y_train_5, y_train_pred_90)
>>> recall_at_90_precision
0.4799852425751706
```

Tuyệt vời, bạn có một bộ phân loại với độ chính xác 90%! Như bạn có thể thấy, việc tạo ra một bộ phân loại với độ chính xác hầu như bất kỳ nào bạn muốn là khá dễ dàng: chỉ cần đặt ngưỡng đủ cao, và bạn đã hoàn thành. Nhưng chờ đã, đừng vội vàng – một bộ phân loại có độ chính xác cao sẽ không hữu ích lắm nếu độ nhạy của nó quá thấp! Đối với nhiều ứng dụng, độ nhạy 48% sẽ không tốt chút nào.

3.3.5 Đường cong ROC

Đường cong đặc trưng hoạt động của bộ thu (ROC) là một công cụ phổ biến khác được sử dụng với các bộ phân loại nhị phân. Nó rất giống với đường cong độ chính xác/độ nhạy, nhưng thay vì vẽ biểu đồ độ chính xác so với độ nhạy, đường cong ROC vẽ biểu đồ tỷ lệ dương tính đúng (một tên khác của độ nhạy) so với tỷ lệ dương tính giả (FPR).

FPR (còn được gọi là fall-out) là tỷ lệ các trường hợp âm tính bị phân loại sai là dương tính. Nó bằng 1 trừ đi tỷ lệ âm tính đúng (TNR), là tỷ lệ các trường hợp âm tính được phân loại đúng là âm tính. TNR cũng được gọi là độ đặc hiệu (specificity). Do đó, đường cong ROC vẽ biểu đồ độ nhạy (recall) so với $1 - \text{độ đặc hiệu}$.

Để vẽ đường cong ROC, trước tiên bạn sử dụng hàm `roc_curve()` để tính toán TPR và FPR cho các giá trị ngưỡng khác nhau:

```
from sklearn.metrics import roc_curve

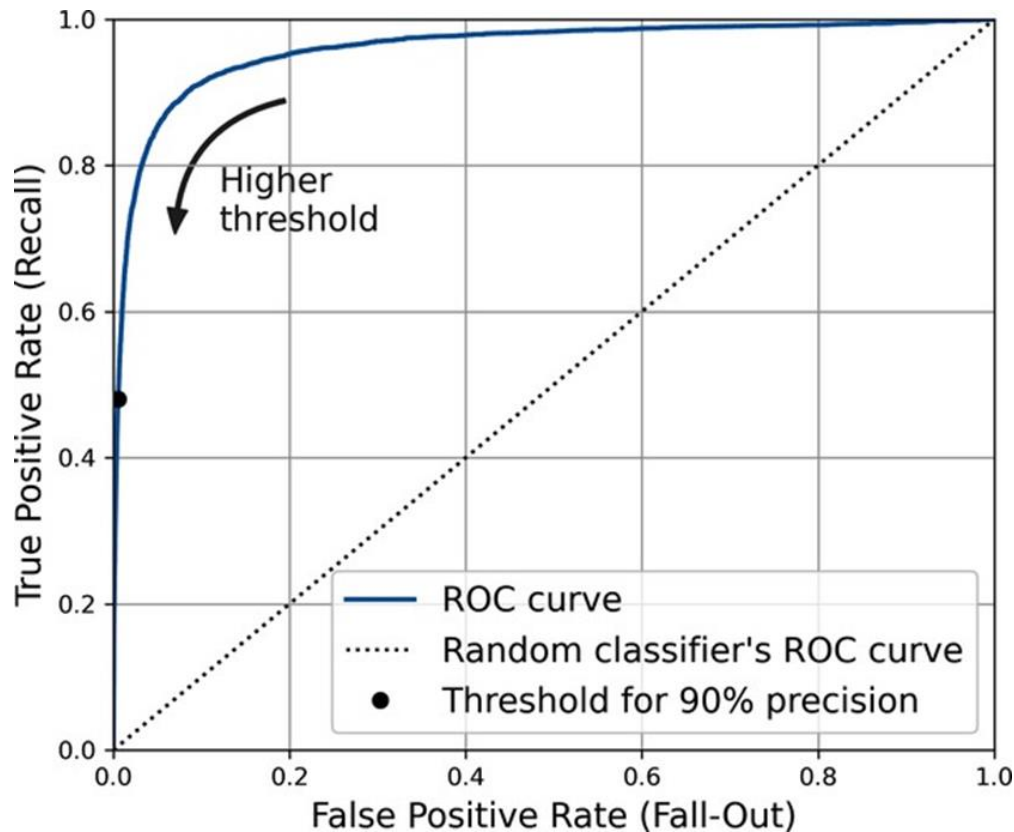
fpr, tpr, thresholds = roc_curve(y_train_5, y_scores)
```

Sau đó, bạn có thể vẽ FPR so với TPR bằng Matplotlib. Đoạn mã sau tạo ra biểu đồ trong Hình 3-7.

Để tìm điểm tương ứng với độ chính xác 90%, chúng ta cần tìm chỉ mục của ngưỡng mong muốn. Vì các ngưỡng được liệt kê theo thứ tự giảm dần trong trường hợp này, chúng ta sử dụng `<=` thay vì `>=` trên dòng đầu tiên:

```
idx_for_threshold_at_90 = (thresholds <= threshold_for_90_precision).argmax()
tpr_90, fpr_90 = tpr[idx_for_threshold_at_90], fpr[idx_for_threshold_at_90]
```

```
plt.plot(fpr, tpr, linewidth=2, label="ROC curve")
plt.plot([0, 1], [0, 1], 'k:', label="Random classifier's ROC curve")
plt.plot([fpr_90], [tpr_90], "ko", label="Threshold for 90% precision")
[...] # beautify the figure: add labels, grid, legend, arrow, and text
plt.show()
```



Hình 3-7. Đường cong ROC vẽ tỷ lệ dương tính giả so với tỷ lệ dương tính đúng cho tất cả các ngưỡng có thể; vòng tròn đen làm nổi bật tỷ lệ đã chọn (ở độ chính xác 90% và độ nhạy 48%).

Một lần nữa lại có một sự đánh đổi: độ nhạy (TPR) càng cao, bộ phân loại tạo ra càng nhiều dương tính giả (FPR). Đường chấm chấm đại diện cho đường cong ROC của một bộ phân loại hoàn toàn ngẫu nhiên; một bộ phân loại tốt nằm càng xa đường đó càng tốt (hướng về góc trên bên trái).

Một cách để so sánh các bộ phân loại là đo diện tích dưới đường cong (AUC). Một bộ phân loại hoàn hảo sẽ có ROC AUC bằng 1, trong khi một bộ phân loại hoàn toàn ngẫu nhiên sẽ có ROC AUC bằng 0.5. Scikit-Learn cung cấp một hàm để ước tính ROC AUC:

```
>>> from sklearn.metrics import roc_auc_score

>>> roc_auc_score(y_train_5, y_scores)
0.9604938554008616
```

Bây giờ hãy tạo một `RandomForestClassifier`, mà chúng ta có thể so sánh đường cong PR và điểm F1 của nó với `SGDClassifier`.

```
from sklearn.ensemble import RandomForestClassifier
```

```
forest_clf = RandomForestClassifier(random_state=42)
```

Hàm `precision_recall_curve()` mong đợi nhãn và điểm cho mỗi trường hợp, vì vậy chúng ta cần huấn luyện bộ phân loại rừng ngẫu nhiên và yêu cầu nó gán điểm cho mỗi trường hợp. Nhưng lớp `RandomForestClassifier` không có phương thức `decision_function()`, do cách nó hoạt động. May mắn thay, nó có phương thức `predict_proba()` trả về xác suất lớp cho mỗi trường hợp, và chúng ta có thể chỉ cần sử dụng xác suất của lớp dương tính làm điểm, vì vậy nó sẽ hoạt động tốt. Chúng ta có thể gọi hàm `cross_val_predict()` để huấn luyện `RandomForestClassifier` bằng cách sử dụng kiểm định chéo và yêu cầu nó dự đoán xác suất lớp cho mọi hình ảnh như sau:

```
y_probas_forest = cross_val_predict(forest_clf, X_train, y_train_5, cv=3,
                                    method="predict_proba")
```

Hãy xem xác suất lớp cho hai hình ảnh đầu tiên trong tập huấn luyện:

```
>>> y_probas_forest[:2]
array([[0.11, 0.89],
       [0.99, 0.01]])
```

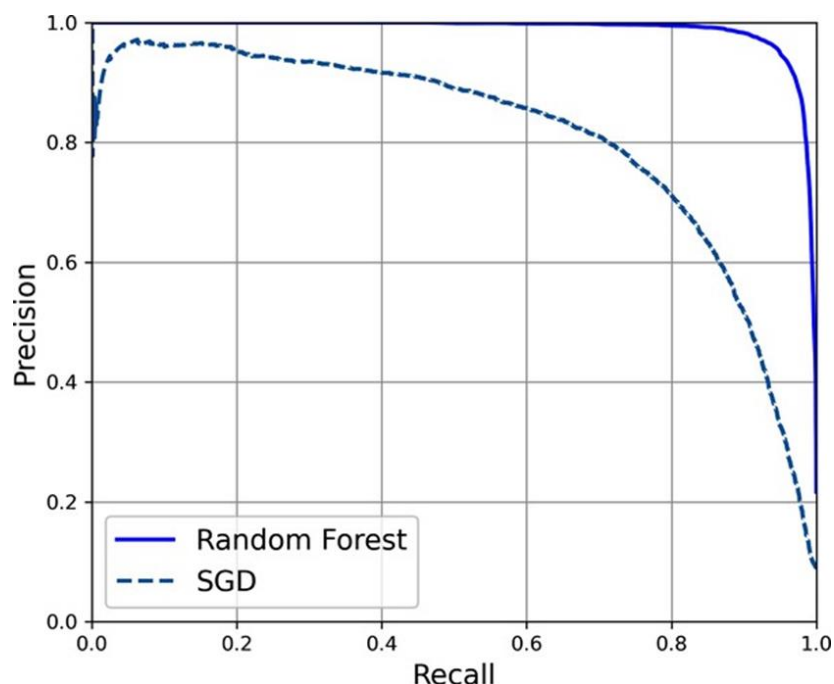
Mô hình dự đoán rằng hình ảnh đầu tiên là tích cực với xác suất 89%, và nó dự đoán rằng hình ảnh thứ hai là tiêu cực với xác suất 99%. Vì mỗi hình ảnh hoặc là tích cực hoặc là tiêu cực, xác suất trong mỗi hàng cộng lại bằng 100%.

Cột thứ hai chứa xác suất ước tính cho lớp dương tính, vì vậy hãy truyền chúng cho hàm `precision_recall_curve()`:

```
y_scores_forest = y_probas_forest[:, 1]
precisions_forest, recalls_forest, thresholds_forest = \
    precision_recall_curve(y_train_5, y_scores_forest)
```

Bây giờ chúng ta đã sẵn sàng vẽ đường cong PR. Việc vẽ đường cong PR đầu tiên cũng hữu ích để xem chúng so sánh như thế nào (Hình 3-8):

```
plt.plot(recalls_forest, precisions_forest, "b-", linewidth=2,
         label="Random Forest")
plt.plot(recalls, precisions, "--", linewidth=2, label="SGD")
[...] # beautify the figure: add labels, grid, and legend
plt.show()
```



Hình 3-8. So sánh đường cong PR: bộ phân loại rừng ngẫu nhiên vượt trội hơn bộ phân loại SGD vì đường cong PR của nó gần góc trên bên phải hơn nhiều, và nó có AUC lớn hơn.

Như bạn có thể thấy trong Hình 3-8, đường cong PR của RandomForestClassifier trông tốt hơn nhiều so với SGDClassifier: nó gần góc trên bên phải hơn nhiều. Điểm F1 và điểm ROC AUC của nó cũng tốt hơn đáng kể:

```
>>> y_train_pred_forest = y_probas_forest[:, 1] >= 0.5 # positive proba ≥ 50%
>>> f1_score(y_train_5, y_train_pred_forest)
0.9242275142688446
>>> roc_auc_score(y_train_5, y_scores_forest)
0.9983436731328145
```

Hãy thử đo điểm độ chính xác và độ nhạy: bạn sẽ tìm thấy khoảng 99.1% độ chính xác và 86.6% độ nhạy. Không tệ chút nào!

Bây giờ bạn đã biết cách huấn luyện các bộ phân loại nhị phân, chọn chỉ số phù hợp cho tác vụ của bạn, đánh giá các bộ phân loại của bạn bằng cách sử dụng kiểm định chéo, chọn sự đánh đổi độ chính xác/độ nhạy phù hợp với nhu cầu của bạn, và sử dụng một số chỉ số và đường cong để so sánh các mô hình khác nhau. Bạn đã sẵn sàng thử phát hiện nhiều hơn chỉ số 5.

3.4 Phân loại đa lớp

Trong khi các bộ phân loại nhị phân phân biệt giữa hai lớp, các bộ phân loại đa lớp (còn được gọi là bộ phân loại đa thức) có thể phân biệt giữa nhiều hơn hai lớp.

Một số bộ phân loại của Scikit-Learn (ví dụ: LogisticRegression, RandomForestClassifier và GaussianNB) có khả năng xử lý nhiều lớp một cách tự nhiên. Các bộ phân loại khác lại là

bộ phân loại nhị phân nghiêm ngặt (ví dụ: `SGDClassifier` và `SVC`). Tuy nhiên, có nhiều chiến lược khác nhau mà bạn có thể sử dụng để thực hiện phân loại đa lớp với nhiều bộ phân loại nhị phân.

Một cách để tạo một hệ thống có thể phân loại hình ảnh chữ số thành 10 lớp (từ 0 đến 9) là huấn luyện 10 bộ phân loại nhị phân, mỗi bộ cho một chữ số (một bộ phát hiện số 0, một bộ phát hiện số 1, một bộ phát hiện số 2, v.v.). Sau đó, khi bạn muốn phân loại một hình ảnh, bạn sẽ lấy điểm quyết định từ mỗi bộ phân loại cho hình ảnh đó và chọn lớp có bộ phân loại xuất ra điểm cao nhất. Điều này được gọi là chiến lược một-đối-phần-còn-lại (OvR), hoặc đôi khi là một-đối-tất-cả (OvA).

Một chiến lược khác là huấn luyện một bộ phân loại nhị phân cho mỗi cặp chữ số: một để phân biệt số 0 và số 1, một để phân biệt số 0 và số 2, một để phân biệt số 1 và số 2, v.v. Điều này được gọi là chiến lược một-đối-một (OvO). Nếu có N lớp, bạn cần huấn luyện $N \times (N - 1) / 2$ bộ phân loại. Đối với vấn đề MNIST, điều này có nghĩa là huấn luyện 45 bộ phân loại nhị phân! Khi bạn muốn phân loại một hình ảnh, bạn phải chạy hình ảnh đó qua tất cả 45 bộ phân loại và xem lớp nào thắng nhiều trận đấu nhất. Ưu điểm chính của OvO là mỗi bộ phân loại chỉ cần được huấn luyện trên phần tập huấn luyện chứa hai lớp mà nó phải phân biệt.

Một số thuật toán (chẳng hạn như bộ phân loại máy vector hỗ trợ) mở rộng kém với kích thước của tập huấn luyện. Đối với các thuật toán này, OvO được ưu tiên vì huấn luyện nhiều bộ phân loại trên các tập huấn luyện nhỏ nhanh hơn so với huấn luyện ít bộ phân loại trên các tập huấn luyện lớn. Tuy nhiên, đối với hầu hết các thuật toán phân loại nhị phân, OvR được ưu tiên hơn.

Scikit-Learn phát hiện khi bạn cố gắng sử dụng thuật toán phân loại nhị phân cho tác vụ phân loại đa lớp, và nó tự động chạy OvR hoặc OvO, tùy thuộc vào thuật toán. Hãy thử điều này với bộ phân loại máy vector hỗ trợ sử dụng lớp `sklearn.svm.SVC` (xem Chương 5). Chúng ta sẽ chỉ huấn luyện trên 2.000 hình ảnh đầu tiên, nếu không sẽ mất rất nhiều thời gian:

```
from sklearn.svm import SVC

svm_clf = SVC(random_state=42)
svm_clf.fit(X_train[:2000], y_train[:2000]) # y_train, not y_train_5
```

Thật dễ dàng! Chúng ta đã huấn luyện SVC bằng cách sử dụng các lớp mục tiêu gốc từ 0 đến 9 (`y_train`), thay vì các lớp mục tiêu 5-đối-phần-còn-lại (`y_train_5`).

Vì có 10 lớp (tức là hơn 2), Scikit-Learn đã sử dụng chiến lược OvO và huấn luyện 45 bộ phân loại nhị phân. Bây giờ hãy đưa ra dự đoán trên một hình ảnh:

```
>>> svm_clf.predict([some_digit])
array(['5'], dtype=object)
```

Đúng rồi! Mã này thực sự đã đưa ra 45 dự đoán — một cho mỗi cặp lớp — và nó đã chọn lớp thắng nhiều “trận đấu” nhất. Nếu bạn gọi phương thức `decision_function()`, bạn sẽ thấy nó trả về 10 điểm cho mỗi trường hợp: một cho mỗi lớp. Mỗi lớp nhận được một điểm bằng

số “trận đấu” thắng được cộng hoặc trừ một điều chỉnh nhỏ (tối đa ± 0.33) để phá vỡ các trường hợp hòa, dựa trên điểm số của bộ phân loại:

```
>>> some_digit_scores = svm_clf.decision_function([some_digit])
>>> some_digit_scores.round(2)
array([[ 3.79,  0.73,  6.06,  8.3 , -0.29,  9.3 ,  1.75,  2.77,  7.21,
         4.82]])
```

Điểm cao nhất là 9.3, và đó thực sự là điểm tương ứng với lớp 5:

```
>>> class_id = some_digit_scores.argmax()
>>> class_id
5
```

Khi một bộ phân loại được huấn luyện, nó lưu trữ danh sách các lớp mục tiêu trong thuộc tính `classes_` của nó, được sắp xếp theo giá trị. Trong trường hợp MNIST, chỉ mục của mỗi lớp trong mảng `classes_` trùng khớp với chính lớp đó (ví dụ: lớp ở chỉ mục 5 thì chính là lớp ‘5’), nhưng nói chung bạn sẽ không may mắn như vậy; bạn sẽ cần tra cứu nhãn lớp như thế này:

```
>>> svm_clf.classes_
array(['0', '1', '2', '3', '4', '5', '6', '7', '8', '9'], dtype=object)
>>> svm_clf.classes_[class_id]
'5'
```

Nếu bạn muốn buộc Scikit-Learn sử dụng chiến lược một-đối-một hoặc một-đối-phần-còn-lại, bạn có thể sử dụng các lớp `OneVsOneClassifier` hoặc `OneVsRestClassifier`. Chỉ cần tạo một thể hiện và truyền một bộ phân loại vào hàm tạo của nó (nó thậm chí không cần phải là bộ phân loại nhị phân). Ví dụ, mã này tạo một bộ phân loại đa lớp sử dụng chiến lược OvR, dựa trên một SVC:

```
from sklearn.multiclass import OneVsRestClassifier

ovr_clf = OneVsRestClassifier(SVC(random_state=42))
ovr_clf.fit(X_train[:2000], y_train[:2000])
```

Hãy đưa ra dự đoán và kiểm tra số lượng bộ phân loại đã được huấn luyện:

```
>>> ovr_clf.predict([some_digit])
array(['5'], dtype='<U1')
>>> len(ovr_clf.estimators_)
10
```

Huấn luyện một `SGDClassifier` trên một tập dữ liệu đa lớp và sử dụng nó để đưa ra dự đoán cũng dễ dàng như vậy:

```
>>> sgd_clf = SGDClassifier(random_state=42)
>>> sgd_clf.fit(X_train, y_train)
>>> sgd_clf.predict([some_digit])
array(['3'], dtype='<U1')
```

Ồi, sai rồi. Lỗi dự đoán vẫn xảy ra! Lần này Scikit-Learn đã sử dụng chiến lược OvR ẩn bên trong: vì có 10 lớp, nó đã huấn luyện 10 bộ phân loại nhị phân. Phương thức `decision_function()` bây giờ trả về một giá trị cho mỗi lớp. Hãy xem các điểm mà bộ phân loại SGD đã gán cho mỗi lớp:

```
>>> sgd_clf.decision_function([some_digit]).round()
array([[ -31893.,  -34420.,  -9531.,   1824., -22320.,  -1386., -26189.,
        -16148.,  -4604., -12051.]])
```

Bạn có thể thấy rằng bộ phân loại không tự tin lắm về dự đoán của mình: hầu hết các điểm đều rất âm, trong khi lớp 3 có điểm +1.824, và lớp 5 không quá xa phía sau ở -1.386. Tất nhiên, bạn sẽ muốn đánh giá bộ phân loại này trên nhiều hơn một hình ảnh. Vì có khoảng cùng số lượng hình ảnh trong mỗi lớp, chỉ số độ chính xác là ổn. Như thường lệ, bạn có thể sử dụng hàm `cross_val_score()` để đánh giá mô hình:

```
>>> cross_val_score(sgd_clf, X_train, y_train, cv=3, scoring="accuracy")
array([0.87365, 0.85835, 0.8689 ])
```

Nó đạt hơn 85.8% trên tất cả các fold kiểm thử. Nếu bạn sử dụng một bộ phân loại ngẫu nhiên, bạn sẽ đạt độ chính xác 10%, vì vậy đây không phải là một điểm số quá tệ, nhưng bạn vẫn có thể làm tốt hơn nhiều. Đơn giản là điều chỉnh tỷ lệ đầu vào (như đã thảo luận trong Chương 2) làm tăng độ chính xác lên trên 89.1%:

```
from sklearn.preprocessing import StandardScaler

>>> scaler = StandardScaler()
>>> X_train_scaled = scaler.fit_transform(X_train.astype("float64"))
>>> cross_val_score(sgd_clf, X_train_scaled, y_train, cv=3, scoring="accuracy")
array([0.8983, 0.891 , 0.9018])
```

3.5 Phân tích lỗi

Nếu đây là một dự án thực tế, bây giờ bạn sẽ thực hiện các bước trong danh sách kiểm tra dự án học máy của mình (xem Phụ lục A). Bạn sẽ khám phá các tùy chọn chuẩn bị dữ liệu, thử nhiều mô hình, chọn lọc những mô hình tốt nhất, tinh chỉnh siêu tham số của chúng bằng `GridSearchCV` và tự động hóa càng nhiều càng tốt. Ở đây, chúng ta sẽ giả định rằng bạn đã tìm thấy một mô hình hứa hẹn và bạn muốn tìm cách cải thiện nó. Một cách để làm điều này là phân tích các loại lỗi mà nó mắc phải.

Đầu tiên, hãy xem ma trận nhầm lẫn. Để làm điều này, trước tiên bạn cần tạo dự đoán bằng hàm `cross_val_predict()`; sau đó bạn có thể truyền nhãn và dự đoán cho hàm `confusion_matrix()`, giống như bạn đã làm trước đây. Tuy nhiên, vì bây giờ có 10 lớp thay vì 2, ma trận nhầm lẫn sẽ chứa khá nhiều số, và có thể khó đọc.

Một biểu đồ màu của ma trận nhầm lẫn dễ phân tích hơn nhiều. Để vẽ biểu đồ như vậy, hãy sử dụng hàm `ConfusionMatrixDisplay.from_predictions()` như sau:

```
from sklearn.metrics import ConfusionMatrixDisplay

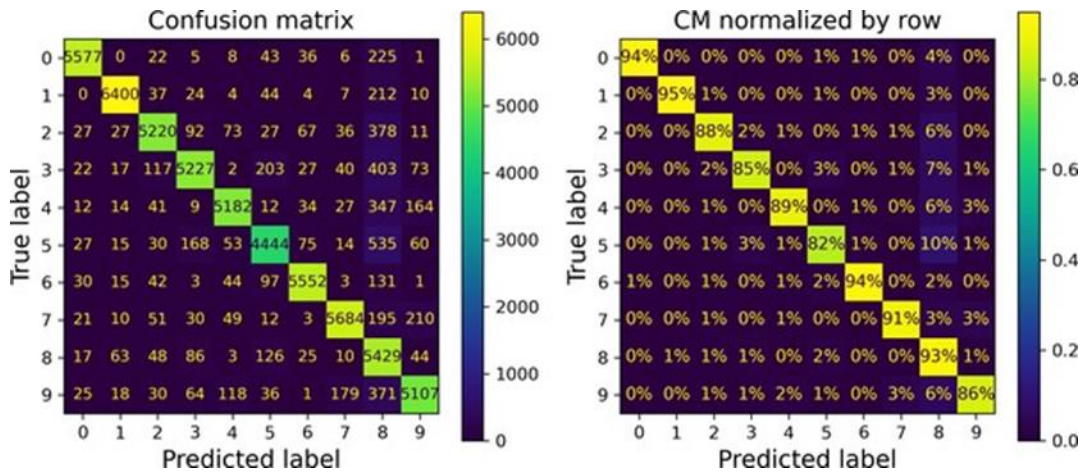
y_train_pred = cross_val_predict(sgd_clf, X_train_scaled, y_train, cv=3)
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred)
plt.show()
```

Điều này tạo ra biểu đồ bên trái trong Hình 3-9. Ma trận nhầm lẫn này trông khá tốt: hầu hết các hình ảnh nằm trên đường chéo chính, có nghĩa là chúng được phân loại đúng. Lưu ý rằng ô trên đường chéo ở hàng #5 và cột #5 trông hơi tối hơn các chữ số khác. Điều này có thể là do mô hình mắc nhiều lỗi hơn với các số 5, hoặc vì có ít số 5 hơn trong tập dữ liệu so với các chữ số khác. Đó là lý do tại sao điều quan trọng là phải chuẩn hóa ma trận nhầm lẫn bằng cách chia mỗi giá trị cho tổng số hình ảnh trong lớp (thực tế) tương ứng (tức là chia cho tổng hàng). Điều này có thể được thực hiện đơn giản bằng cách đặt `normalize="true"`. Chúng ta cũng có thể chỉ định đối số `values_format=".0%"` để hiển thị phần trăm không có số thập phân. Đoạn mã sau tạo ra biểu đồ bên phải trong Hình 3-9:

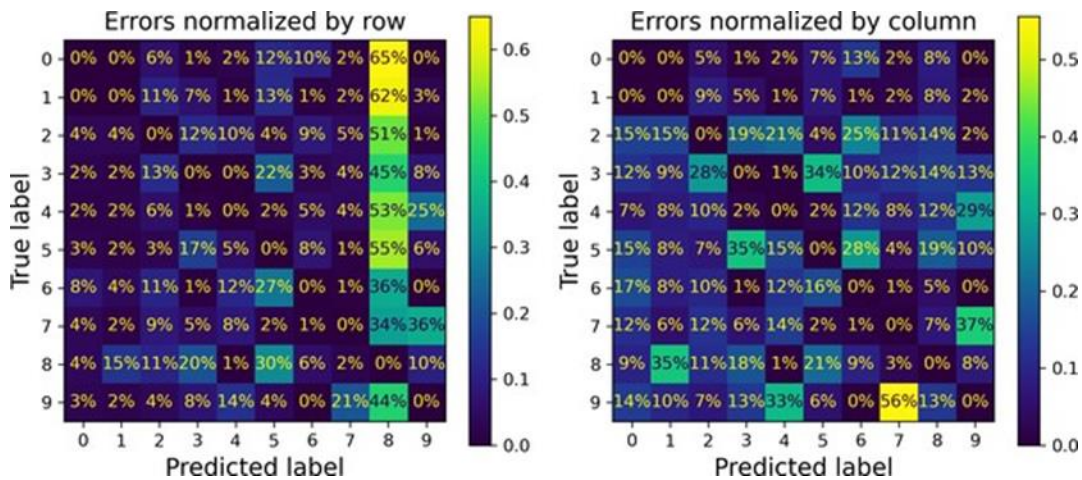
```
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred,
                                       normalize="true",
                                       values_format=".0%")
plt.show()
```

Bây giờ chúng ta có thể dễ dàng thấy rằng chỉ có 82% hình ảnh số 5 được phân loại đúng. Lỗi phổ biến nhất mà mô hình mắc phải với hình ảnh số 5 là phân loại sai chúng thành số 8: điều này xảy ra với 10% tổng số số 5. Nhưng chỉ 2% số 8 bị phân loại sai thành số 5; ma trận nhầm lẫn nói chung không đối xứng! Nếu bạn nhìn kỹ, bạn sẽ nhận thấy rằng nhiều chữ số đã bị phân loại sai thành số 8, nhưng điều này không hiển thị rõ ngay lập tức từ biểu đồ này. Nếu bạn muốn làm cho các lỗi nổi bật hơn, bạn có thể thử đặt trọng số 0 cho các dự đoán đúng. Đoạn mã sau đây thực hiện điều đó và tạo ra biểu đồ bên trái trong Hình 3-10:

```
sample_weight = (y_train_pred != y_train)
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred,
                                       sample_weight=sample_weight,
                                       normalize="true",
                                       values_format=".0%")
plt.show()
```



Hình 3-9. Ma trận nhầm lẫn (trái) và cùng ma trận nhầm lẫn được chuẩn hóa theo hàng (phải).



Hình 3-10. Ma trận nhầm lẫn chỉ hiển thị lỗi, được chuẩn hóa theo hàng (trái) và theo cột (phải).

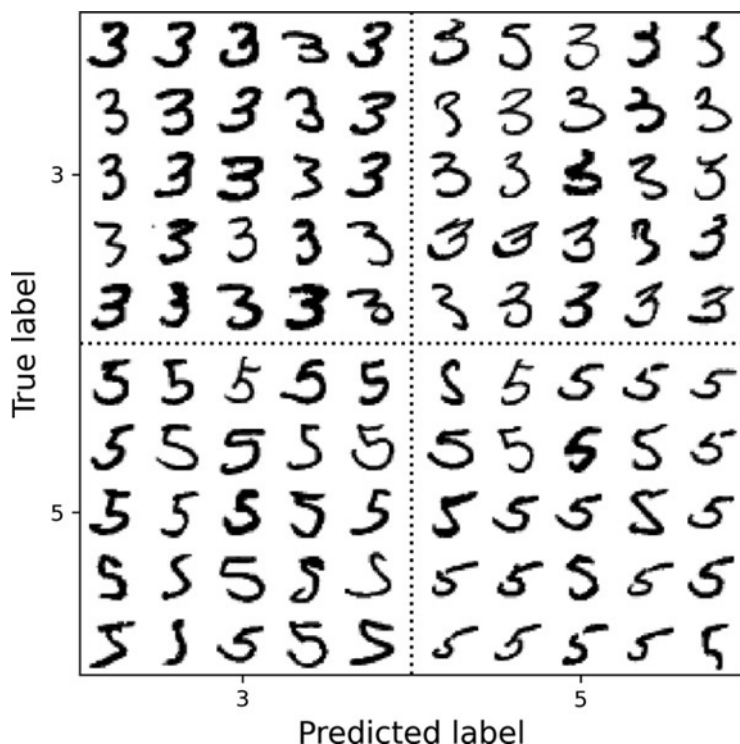
Bây giờ bạn có thể thấy rõ hơn nhiều các loại lỗi mà bộ phân loại mắc phải. Cột dành cho lớp 8 bây giờ thực sự sáng, điều này xác nhận rằng nhiều hình ảnh bị phân loại sai là số 8. Trên thực tế, đây là lỗi phân loại sai phổ biến nhất đối với hầu hết các lớp. Nhưng hãy cẩn thận cách bạn diễn giải tỷ lệ phần trăm trong biểu đồ này: hãy nhớ rằng chúng ta đã loại trừ các dự đoán đúng. Ví dụ, 36% ở hàng #7, cột #9 không có nghĩa là 36% tất cả các hình ảnh số 7 bị phân loại sai là số 9. Nó có nghĩa là 36% các lỗi mà mô hình mắc phải trên hình ảnh số 7 là phân loại sai là số 9. Trong thực tế, chỉ 3% hình ảnh số 7 bị phân loại sai là số 9, như bạn có thể thấy trong biểu đồ bên phải trong Hình 3-9.

Cũng có thể chuẩn hóa ma trận nhầm lẫn theo cột thay vì theo hàng: nếu bạn đặt `normalize="pred"`, bạn sẽ nhận được biểu đồ ở bên phải trong Hình 3-10. Ví dụ, bạn có thể thấy rằng 56% số 7 bị phân loại sai thực sự là số 9.

Phân tích ma trận nhầm lẫn thường cung cấp cho bạn cái nhìn sâu sắc về các cách để cải thiện bộ phân loại của bạn. Nhìn vào các biểu đồ này, có vẻ như nỗ lực của bạn nên tập trung vào việc giảm các số 8 sai. Ví dụ, bạn có thể thử thu thập thêm dữ liệu huấn luyện cho các chữ số trông giống số 8 (nhưng không phải) để bộ phân loại có thể học cách phân biệt chúng với số 8 thực. Hoặc bạn có thể thiết kế các đặc trưng mới sẽ giúp bộ phân loại — ví dụ, viết một thuật toán để đếm số vòng tròn kín (ví dụ: số 8 có hai, số 6 có một, số 5 không có). Hoặc bạn có thể tiền xử lý hình ảnh (ví dụ: sử dụng Scikit-Image, Pillow hoặc OpenCV) để làm cho một số mẫu, chẳng hạn như vòng tròn kín, nổi bật hơn.

Phân tích các lỗi riêng lẻ cũng có thể là một cách tốt để hiểu rõ hơn về những gì bộ phân loại của bạn đang làm và tại sao nó lại thất bại. Ví dụ, hãy vẽ biểu đồ các ví dụ về số 3 và số 5 theo kiểu ma trận nhầm lẫn (Hình 3-11):

```
cl_a, cl_b = '3', '5'
X_aa = X_train[(y_train == cl_a) & (y_train_pred == cl_a)]
X_ab = X_train[(y_train == cl_a) & (y_train_pred == cl_b)]
X_ba = X_train[(y_train == cl_b) & (y_train_pred == cl_a)]
X_bb = X_train[(y_train == cl_b) & (y_train_pred == cl_b)]
[...] # plot all images in X_aa, X_ab, X_ba, X_bb in a confusion matrix style
```



Hình 3-11. Một số hình ảnh số 3 và số 5 được tổ chức như một ma trận nhầm lẫn.

Như bạn có thể thấy, một số chữ số mà bộ phân loại mắc lỗi (tức là trong các khối dưới cùng bên trái và trên cùng bên phải) được viết quá tệ đến nỗi ngay cả con người cũng gặp khó khăn khi phân loại chúng. Tuy nhiên, hầu hết các hình ảnh bị phân loại sai dường như là lỗi hiển nhiên đối với chúng ta. Có thể khó hiểu tại sao bộ phân loại lại mắc lỗi, nhưng hãy nhớ rằng bộ não con người là một hệ thống nhận dạng mẫu tuyệt vời, và hệ thống thị giác của chúng

ta thực hiện rất nhiều tiền xử lý phức tạp trước khi bất kỳ thông tin nào đến được ý thức của chúng ta. Vì vậy, việc tác vụ này có vẻ đơn giản không có nghĩa là nó thực sự đơn giản. Hãy nhớ rằng chúng ta đã sử dụng một `SGDClassifier` đơn giản, đây chỉ là một mô hình tuyến tính: tất cả những gì nó làm là gán một trọng số cho mỗi lớp cho mỗi pixel, và khi nó thấy một hình ảnh mới, nó chỉ đơn giản là tổng hợp cường độ pixel có trọng số để có được điểm số cho mỗi lớp. Vì số 3 và số 5 chỉ khác nhau vài pixel, mô hình này sẽ dễ dàng nhầm lẫn chúng.

Sự khác biệt chính giữa số 3 và số 5 là vị trí của đường nhỏ nối đường trên cùng với cung dưới cùng. Nếu bạn vẽ số 3 với điểm nối hơi lệch sang trái, bộ phân loại có thể phân loại nó là số 5, và ngược lại. Nói cách khác, bộ phân loại này khá nhạy cảm với sự dịch chuyển và xoay hình ảnh.

Một cách để giảm nhầm lẫn giữa số 3/5 là tiền xử lý hình ảnh để đảm bảo chúng được căn giữa tốt và không bị xoay quá nhiều. Tuy nhiên, điều này có thể không dễ dàng vì nó yêu cầu dự đoán hướng xoay đúng của mỗi hình ảnh. Một cách tiếp cận đơn giản hơn nhiều là tăng cường tập huấn luyện bằng cách thêm các biến thể hình ảnh huấn luyện đã bị dịch chuyển và xoay nhẹ. Điều này sẽ buộc mô hình học cách chịu đựng tốt hơn các biến thể như vậy. Điều này được gọi là tăng cường dữ liệu (data augmentation) (chúng ta sẽ đề cập đến điều này trong Chương 14; cũng xem bài tập 2 ở cuối chương này).

3.6 Phân loại đa nhãn

Cho đến nay, mỗi trường hợp luôn được gán cho chỉ một lớp. Nhưng trong một số trường hợp, bạn có thể muốn bộ phân loại của mình xuất ra nhiều lớp cho mỗi trường hợp. Hãy xem xét một bộ phân loại nhận dạng khuôn mặt: nó nên làm gì nếu nó nhận ra một số người trong cùng một bức ảnh? Nó nên gán một thẻ cho mỗi người mà nó nhận ra. Giả sử bộ phân loại đã được huấn luyện để nhận dạng ba khuôn mặt: Alice, Bob và Charlie. Sau đó, khi bộ phân loại được hiển thị một bức ảnh của Alice và Charlie, nó sẽ xuất ra `[True, False, True]` (nghĩa là “Alice có, Bob không, Charlie có”). Một hệ thống phân loại xuất ra nhiều thẻ nhị phân như vậy được gọi là hệ thống phân loại đa nhãn.

Chúng ta sẽ chưa đi sâu vào nhận dạng khuôn mặt, nhưng hãy xem một ví dụ đơn giản hơn, chỉ để minh họa:

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier

# Giả định X_train và y_train đã được định nghĩa từ dữ liệu MNIST hoặc tương
tự
# Ví dụ:
# from sklearn.datasets import fetch_openml
# mnist = fetch_openml('mnist_784', as_frame=False)
# X, y = mnist.data, mnist.target
# X_train = X[:60000]
# y_train = y[:60000]

y_train_large = (y_train >= '7')
```

```
y_train_odd = (y_train.astype('int8') % 2 == 1)
y_multilabel = np.c_[y_train_large, y_train_odd]
```

```
knn_clf = KNeighborsClassifier()
knn_clf.fit(X_train, y_multilabel)
```

Đoạn mã này tạo ra một mảng `y_multilabel` chứa hai nhãn mục tiêu cho mỗi hình ảnh chữ số: nhãn đầu tiên cho biết chữ số đó có lớn (7, 8 hoặc 9) hay không, và nhãn thứ hai cho biết chữ số đó có lẻ hay không. Sau đó, đoạn mã tạo một thể hiện `KNeighborsClassifier`, hỗ trợ phân loại đa nhãn (không phải tất cả các bộ phân loại đều hỗ trợ), và huấn luyện mô hình này bằng cách sử dụng mảng mục tiêu đa nhãn. Bây giờ bạn có thể đưa ra dự đoán và nhận thấy rằng nó xuất ra hai nhãn:

```
>>> knn_clf.predict([some_digit]) # some_digit là một ví dụ từ tập X_train
array([[False,  True]])
```

Và nó đoán đúng! Chữ số 5 thực sự không lớn (`False`) và lẻ (`True`).

Có nhiều cách để đánh giá một bộ phân loại đa nhãn, và việc chọn chỉ số phù hợp thực sự phụ thuộc vào dự án của bạn. Một cách tiếp cận là đo điểm F1 cho từng nhãn riêng lẻ (hoặc bất kỳ chỉ số bộ phân loại nhị phân nào khác đã thảo luận trước đó), sau đó chỉ cần tính điểm trung bình. Đoạn mã sau tính điểm F1 trung bình trên tất cả các nhãn:

```
# Giả định cross_val_predict và f1_score đã được import
# from sklearn.model_selection import cross_val_predict
# from sklearn.metrics import f1_score
y_train_knn_pred = cross_val_predict(knn_clf, X_train, y_multilabel, cv=3)
>>> f1_score(y_multilabel, y_train_knn_pred, average="macro") [cite: 5]
0.976410265560605
```

Cách tiếp cận này giả định rằng tất cả các nhãn đều quan trọng như nhau, điều này có thể không đúng. Đặc biệt, nếu bạn có nhiều ảnh của Alice hơn Bob hoặc Charlie, bạn có thể muốn ưu tiên điểm số của bộ phân loại trên ảnh của Alice. Một lựa chọn đơn giản là gán cho mỗi nhãn một trọng số bằng với *hỗ trợ* của nó (tức là số lượng trường hợp có nhãn mục tiêu đó). Để làm điều này, chỉ cần đặt `average="weighted"` khi gọi hàm `f1_score()`.

Nếu bạn muốn sử dụng một bộ phân loại không hỗ trợ phân loại đa nhãn một cách tự nhiên, chẳng hạn như SVC, một chiến lược khả thi là huấn luyện một mô hình cho mỗi nhãn. Tuy nhiên, chiến lược này có thể gặp khó khăn trong việc nắm bắt các phụ thuộc giữa các nhãn. Ví dụ, một chữ số lớn (7, 8 hoặc 9) có khả năng lẻ gấp đôi so với chẵn, nhưng bộ phân loại cho nhãn “lẻ” không biết bộ phân loại cho nhãn “lớn” đã dự đoán gì. Để giải quyết vấn đề này, các mô hình có thể được tổ chức thành một chuỗi: khi một mô hình đưa ra dự đoán, nó sử dụng các đặc trưng đầu vào cộng với tất cả các dự đoán của các mô hình đi trước nó trong chuỗi.

Tin tốt là Scikit-Learn có một lớp tên là `ChainClassifier` làm được điều đó! Theo mặc định, nó sẽ sử dụng các nhãn đúng để huấn luyện, cung cấp cho mỗi mô hình các nhãn thích hợp tùy thuộc vào vị trí của chúng trong chuỗi. Nhưng nếu bạn đặt siêu tham số `cv`, nó sẽ sử dụng

kiểm định chéo để có được các dự đoán “sạch” (ngoài mẫu) từ mỗi mô hình đã huấn luyện cho mọi trường hợp trong tập huấn luyện, và những dự đoán này sau đó sẽ được sử dụng để huấn luyện tất cả các mô hình sau đó trong chuỗi. Dưới đây là một ví dụ cho thấy cách tạo và huấn luyện ChainClassifier bằng cách sử dụng chiến lược kiểm định chéo. Như trước, chúng ta sẽ chỉ sử dụng 2.000 hình ảnh đầu tiên trong tập huấn luyện để tăng tốc:

```
from sklearn.multioutput import ClassifierChain
from sklearn.svm import SVC # Import SVC as it's used in the example

chain_clf = ClassifierChain(SVC(), cv=3, random_state=42)
chain_clf.fit(X_train[:2000], y_multilabel[:2000])
```

Bây giờ chúng ta có thể sử dụng ChainClassifier này để đưa ra dự đoán:

```
>>> chain_clf.predict([some_digit])
array([[0., 1.]])
```

3.7 Phân loại đa đầu ra

Loại tác vụ phân loại cuối cùng chúng ta sẽ thảo luận ở đây được gọi là phân loại đa đầu ra – đa lớp (hoặc chỉ phân loại đa đầu ra). Đây là một khái quát hóa của phân loại đa nhãn, trong đó mỗi nhãn có thể là đa lớp (tức là nó có thể có nhiều hơn hai giá trị có thể).

Để minh họa điều này, hãy xây dựng một hệ thống loại bỏ nhiễu khỏi hình ảnh. Nó sẽ nhận một hình ảnh chữ số bị nhiễu làm đầu vào, và nó sẽ (hy vọng) xuất ra một hình ảnh chữ số sạch, được biểu diễn dưới dạng một mảng cường độ pixel, giống như các hình ảnh MNIST. Lưu ý rằng đầu ra của bộ phân loại là đa nhãn (một nhãn cho mỗi pixel) và mỗi nhãn có thể có nhiều giá trị (cường độ pixel dao động từ 0 đến 255). Do đó, đây là một ví dụ về hệ thống phân loại đa đầu ra.

Hãy bắt đầu bằng cách tạo các tập huấn luyện và kiểm thử bằng cách lấy các hình ảnh MNIST và thêm nhiễu vào cường độ pixel của chúng bằng hàm `randint()` của NumPy. Các hình ảnh mục tiêu sẽ là các hình ảnh gốc:

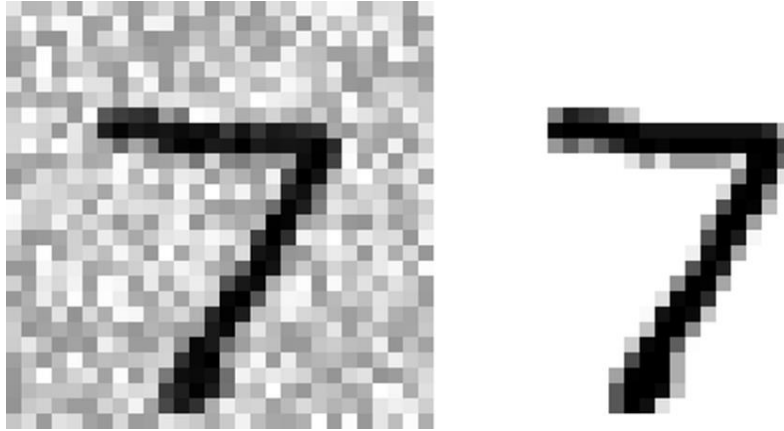
```
import numpy as np
# Giả định X_train, X_test đã được định nghĩa từ dữ liệu MNIST hoặc tương tự
# Ví dụ:
# from sklearn.datasets import fetch_openml
# mnist = fetch_openml('mnist_784', as_frame=False)
# X, y = mnist.data, mnist.target
# X_train = X[:60000]
# X_test = X[60000:]

np.random.seed(42) # to make this code example reproducible
noise = np.random.randint(0, 100, (len(X_train), 784))
X_train_mod = X_train + noise
noise = np.random.randint(0, 100, (len(X_test), 784))
X_test_mod = X_test + noise
```



```
y_train_mod = X_train
y_test_mod = X_test
```

Hãy xem lướt qua hình ảnh đầu tiên từ tập kiểm thử (Hình 3-12). Vâng, chúng ta đang “rình mò” dữ liệu kiểm thử, vì vậy bạn nên nhíu mày ngay bây giờ.

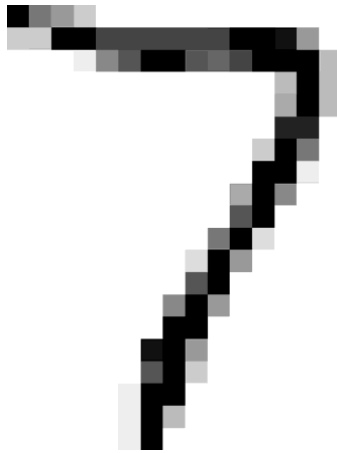


Hình 3-12. Một hình ảnh bị nhiễu (trái) và hình ảnh mục tiêu sạch (phải).

Bên trái là hình ảnh đầu vào bị nhiễu, và bên phải là hình ảnh mục tiêu sạch. Bây giờ hãy huấn luyện bộ phân loại và làm cho nó làm sạch hình ảnh này (Hình 3-13):

```
from sklearn.neighbors import KNeighborsClassifier
# Giả định plot_digit đã được định nghĩa
# import matplotlib.pyplot as plt
# def plot_digit(image_data):
#     image = image_data.reshape(28, 28)
#     plt.imshow(image, cmap="binary")
#     plt.axis("off")

knn_clf = KNeighborsClassifier()
knn_clf.fit(X_train_mod, y_train_mod)
clean_digit = knn_clf.predict([X_test_mod[0]])
plot_digit(clean_digit)
plt.show()
```



Hình 3-13. Hình ảnh đã được làm sạch.

Trông khá giống với mục tiêu! Điều này kết thúc chuyến tham quan của chúng ta về phân loại. Bây giờ bạn đã biết cách chọn các chỉ số tốt cho các tác vụ phân loại, chọn sự đánh đổi độ chính xác/độ nhạy phù hợp, so sánh các bộ phân loại và tổng quát hơn là xây dựng các hệ thống phân loại tốt cho nhiều tác vụ khác nhau. Trong các chương tiếp theo, bạn sẽ tìm hiểu cách tất cả các mô hình học máy mà bạn đã sử dụng thực sự hoạt động.

3.8 Bài tập

- Hãy thử xây dựng một bộ phân loại cho tập dữ liệu MNIST đạt độ chính xác trên 97% trên tập kiểm thử. Gợi ý: `KNeighborsClassifier` hoạt động khá tốt cho tác vụ này; bạn chỉ cần tìm các giá trị siêu tham số tốt (hãy thử tìm kiếm lưới trên các siêu tham số `weights` và `n_neighbors`).
- Viết một hàm có thể dịch chuyển một hình ảnh MNIST theo bất kỳ hướng nào (trái, phải, lên hoặc xuống) một pixel. Sau đó, đối với mỗi hình ảnh trong tập huấn luyện, tạo bốn bản sao đã dịch chuyển (một cho mỗi hướng) và thêm chúng vào tập huấn luyện. Cuối cùng, huấn luyện mô hình tốt nhất của bạn trên tập huấn luyện đã mở rộng này và đo độ chính xác của nó trên tập kiểm thử. Bạn sẽ quan sát thấy rằng mô hình của bạn hoạt động tốt hơn nữa! Kỹ thuật này của việc tăng cường tập huấn luyện một cách nhân tạo được gọi là tăng cường dữ liệu (data augmentation) hoặc mở rộng tập huấn luyện.
- Xử lý tập dữ liệu Titanic. Một nơi tuyệt vời để bắt đầu là trên Kaggle. Ngoài ra, bạn có thể tải dữ liệu từ <https://homl.info/titanic.tgz> và giải nén tarball này giống như bạn đã làm với dữ liệu nhà ở trong Chương 2. Điều này sẽ cung cấp cho bạn hai tệp CSV, `train.csv` và `test.csv`, mà bạn có thể tải bằng `pandas.read_csv()`. Mục tiêu là huấn luyện một bộ phân loại có thể dự đoán cột `Survived` dựa trên các cột khác.
- Xây dựng bộ phân loại thư rác (một bài tập thử thách hơn): a. Tải xuống các ví dụ về thư rác và thư hợp lệ từ các tập dữ liệu công khai của Apache SpamAssassin. b. Giải nén các tập dữ liệu và làm quen với định dạng dữ liệu. c. Chia dữ liệu thành tập huấn luyện và tập kiểm thử. d. Viết một pipeline chuẩn bị dữ liệu để chuyển đổi mỗi email thành một vector đặc trưng. Pipeline chuẩn bị của bạn nên biến đổi một email thành

một vector (thưa) cho biết sự có mặt hoặc vắng mặt của mỗi từ có thể. Ví dụ, nếu tất cả các email chỉ chứa bốn từ, “Hello”, “how”, “are”, “you”, thì email “Hello you Hello Hello you” sẽ được chuyển đổi thành một vector [1, 0, 0, 1] (nghĩa là [“Hello” có mặt, “how” vắng mặt, “are” vắng mặt, “you” có mặt]), hoặc [3, 0, 0, 2] nếu bạn muốn đếm số lần xuất hiện của mỗi từ. Bạn có thể muốn thêm các siêu tham số vào pipeline chuẩn bị của mình để kiểm soát việc có loại bỏ tiêu đề email hay không, chuyển đổi mỗi email thành chữ thường, loại bỏ dấu câu, thay thế tất cả URL bằng “URL”, thay thế tất cả các số bằng “NUMBER”, hoặc thậm chí thực hiện phân tách từ gốc (stemming) (tức là cắt bỏ các đuôi từ; có các thư viện Python có sẵn để làm điều này). e. Cuối cùng, hãy thử một vài bộ phân loại và xem liệu bạn có thể xây dựng một bộ phân loại thư rác tuyệt vời, với cả độ nhạy cao và độ chính xác cao. Các giải pháp cho các bài tập này có sẵn ở cuối sổ ghi chép của chương này, tại <https://homl.info/colab3>.

- Theo mặc định, Scikit-Learn lưu trữ các tập dữ liệu đã tải xuống trong một thư mục có tên `scikit_learn_data` trong thư mục chính của bạn.
- Các tập dữ liệu được trả về bởi `fetch_openml()` không phải lúc nào cũng được xáo trộn hoặc chia.
- Việc xáo trộn có thể là một ý tưởng tồi trong một số ngữ cảnh — ví dụ, nếu bạn đang làm việc với dữ liệu chuỗi thời gian (chẳng hạn như giá thị trường chứng khoán hoặc điều kiện thời tiết). Chúng ta sẽ khám phá điều này trong Chương 15.
- Các bộ phân loại của Scikit-Learn luôn có phương thức `decision_function()` hoặc phương thức `predict_proba()`, hoặc đôi khi cả hai.
- Scikit-Learn cung cấp một vài tùy chọn trung bình hóa khác và các chỉ số bộ phân loại đa nhãn; xem tài liệu để biết thêm chi tiết.
- Bạn có thể sử dụng hàm `shift()` từ mô-đun `scipy.ndimage.interpolation`. Ví dụ, `shift(image, [2, 1], cval=0)` dịch chuyển hình ảnh xuống hai pixel và sang phải một pixel.